

characters) can be easily read. Thereafter, characters again start scrolling rightwards, so the entire message goes out and disappears after a while to reappear from left side.

All the messages are stored in the form of a look-up table in the program memory (ROM) itself. When the circuit is switched 'on' (or reset), the monitoring program first checks for the binary number present at the selection bus and according to that, the ROM address of the starting character of the selected message is loaded into the data-pointer. Thereafter, on-chip ROM reading is used to read the entire message over there.

Note that each character is represented in the look-up table of the source code by two bytes. For exam-

ple, 'S' is represented by 'Sh' and 'Sf' separated by a comma. In addition to the alphabets, Arabic numerals and a few special characters have been defined in the program. For instance, a blank space is represented by 'bsh, bsl.' Thus, it is very easy to modify the program.

Suppose you want to display "HOUSE NO 401-H" in place of message '0.' First, open the source code in the editor. Delete the old string and write the new string as below:

```
msg0: db Hh, Hl, Oh, Ol, Uh, Ul, Sh,
      Sl, Eh, El, bsh, bsl, Nh, Nl, Oh,
      Ol, bsh, bsl, fourh, fourl, zeroh,
      zerol, oneh, onel, msh, mal, Hh, Hl, 0
```

(Please note that the assembler is case-insensitive. Still, upper and lower cases have been used for clarity.)

Future enhancements

Many more messages would be possible if complete Port-3 is used for message selection. Pins RxD, TxD, INT0 and INT1 have been kept free, so that these can be used for interfacing with the serial port of the PC. Also, interrupt pins can be used to display some message and sound an alarm in the case of an emergency. For example, a fire sensor can be connected to 'INT0' and a vibration detector to 'INT1.' These pins can also be used to send signals to synchronise a similar system that displays another related message at the same time, so a 16-character, two-line display is made possible.

EFY note. The source code and other relevant files of this article are included in this month's EFY-CD.

MOVMSG.ASM

```
smod51
DBH equ p2 ; Higher byte of Data
Bus
DBL equ p0 ; Lower byte of Data Bus
ADB equ p1 ; Address Bus
input equ p3 ; message select input

; ** codes for decimal digits are given
below:
; ('h' refers to higher byte, 'l' to
lower one)
zeroh equ 17h
zerol equ 0e8h

oneh equ 0d7h
onel equ 0ffh

twoh equ 23h
twol equ 0e0h

threeh equ 2bh
threel equ 0fch

fourh equ 0c3h
fourl equ 0fbh

fiveh equ 0bh
fivel equ 0f8h

sixh equ 0bh
sixl equ 0e8h

sevenh equ 0d7h
sevenl equ 0fch

eighth equ 03h
eightl equ 0e8h

nineh equ 03h
ninel equ 0f8h

; ** codes for alphabets are given
below:
Ah equ 0c3h
Al equ 0e8h

Bh equ 0bh
Bl equ 0e8h
```

```
Ch equ 3fh
Cl equ 0e8h

Dh equ 03h
Dl equ 0efh

Eh equ 2bh
El equ 0e8h

Fh equ 0ebh
Fl equ 0e8h

Gh equ 1bh
Gl equ 0e8h

Hh equ 0c3h
Hl equ 0ebh

Ih equ 0ffh
Il equ 9fh

Jh equ 17h
Jl equ 0ffh

Kh equ 0e8h
Kl equ 0ebh

Lh equ 3fh
Ll equ 0ebh

Mh equ 0d5h
Ml equ 0e3h

Nh equ 0d6h
Nl equ 0e3h

Oh equ 17h
Ol equ 0e8h

Ph equ 0e3h
Pl equ 0e8h

Qh equ 06h
Ql equ 0e8h

Rh equ 0e2h
Rl equ 0e8h

Sh equ 0bh
```

```
Sl equ 0f8h

Th equ 0ffh
Tl equ 9ch

Uh equ 17h
Ul equ 0ebh

Vh equ 0fdh
Vl equ 6bh

Wh equ 17h
Wl equ 0abh

Xh equ 0fch
Xl equ 77h

Yh equ 0e3h
Yl equ 0bbh

Zh equ 3dh
Zl equ 7ch

; ** codes for few special characters:
strh equ 0e8h ; for star sign (as-
terisk)
strl equ 17h

plsh equ 0ebh ; for '+' sign
plsl equ 9fh

msh equ 0ebh ; minus sign
msl equ 0ffh

_h equ 3fh ; underscore sign
_l equ 0ffh

bsh equ 0ffh ; blank space
bsl equ 0ffh

pieh equ 0eah ; for pie
piel equ 7fh

mueh equ 0e3h ; for micro (mu)
muel equ 0ebh

org 0000h
sjmp main
```